Comprehensive JavaScript Notes: From Basics to Advanced

JavaScript is a versatile, high-level, and interpreted programming language primarily used for creating interactive web pages. It's an essential technology for front-end web development, and increasingly, with Node.js, for back-end development as well.

1. Basic Concepts

1.1 Variables

Variables are containers for storing data values. JavaScript has three keywords for declaring variables: var, let, and const.

- **var**: Oldest way to declare variables. Function-scoped and can be re-declared and re-assigned.
```
var greeting = "Hello";
var greeting = "Hi"; // Re-declaration is allowed
greeting = "Hey";    // Re-assignment is allowed
console.log(greeting); // Output: Hey
```

- **let**: Block-scoped. Can be re-assigned but not re-declared within the same scope.
```
let name = "Alice";
// let name = "Bob"; // Error: Cannot re-declare block-scoped
variable 'name'
name = "Charlie";    // Re-assignment is allowed
console.log(name); // Output: Charlie

if (true) {
    let blockScoped = "I'm inside a block";
    console.log(blockScoped); // Output: I'm inside a block
}
// console.log(blockScoped); // Error: blockScoped is not defined
```

- **const**: Block-scoped. Cannot be re-assigned or re-declared. Must be initialized at declaration.
```
const PI = 3.14159;
// PI = 3.14; // Error: Assignment to constant variable.
// const GRAVITY; // Error: Missing initializer in const
declaration

const person = { name: "David" };
person.name = "Eve"; // Allowed: Modifying properties of an object
declared with const is fine
// person = { name: "Frank" }; // Error: Assignment to constant
```

```
variable.
console.log(person); // Output: { name: "Eve" }
```

1.2 Data Types

JavaScript has two main categories of data types: Primitive and Non-Primitive (Reference).

Primitive Data Types:

1. **String**: Represents text.
   ```
   let message = "Hello, JavaScript!";
   console.log(typeof message); // Output: string
   ```

2. **Number**: Represents both integers and floating-point numbers.
   ```
   let age = 30;
   let price = 99.99;
   console.log(typeof age);   // Output: number
   console.log(typeof price); // Output: number
   ```

3. **Boolean**: Represents true or false.
   ```
   let isActive = true;
   let hasPermission = false;
   console.log(typeof isActive); // Output: boolean
   ```

4. **Undefined**: A variable that has been declared but not yet assigned a value.
   ```
   let undefinedVar;
   console.log(undefinedVar);    // Output: undefined
   console.log(typeof undefinedVar); // Output: undefined
   ```

5. **Null**: Represents the intentional absence of any object value. It's a primitive value.
   ```
   let emptyValue = null;
   console.log(emptyValue);    // Output: null
   console.log(typeof emptyValue); // Output: object (This is a
   long-standing bug in JavaScript)
   ```

6. **Symbol (ES6)**: Represents a unique identifier.
   ```
   const id1 = Symbol('id');
   const id2 = Symbol('id');
   console.log(id1 === id2); // Output: false
   console.log(typeof id1);  // Output: symbol
   ```

7. **BigInt (ES2020)**: Represents whole numbers larger than 2^{53} - 1.
   ```
   const bigNumber = 1234567890123456789012345678901234567890n;
   console.log(typeof bigNumber); // Output: bigint
   ```

Non-Primitive (Reference) Data Types:

1. **Object**: A collection of key-value pairs.
```
let person = {
    firstName: "John",
    lastName: "Doe",
    age: 30
};
console.log(typeof person); // Output: object
```

2. **Array**: A special type of object used to store ordered collections of data.
```
let colors = ["red", "green", "blue"];
console.log(typeof colors); // Output: object (Arrays are objects
in JS)
```

3. **Function**: A block of code designed to perform a particular task. Functions are also objects.
```
function greet() {
    console.log("Hello!");
}
console.log(typeof greet); // Output: function (though technically
an object)
```

1.3 Operators

Operators perform operations on values and variables.

- **Arithmetic Operators**: +, -, *, /, % (modulus), ** (exponentiation), ++ (increment), -- (decrement)
```
let a = 10, b = 3;
console.log(a + b);  // 13
console.log(a - b);  // 7
console.log(a * b);  // 30
console.log(a / b);  // 3.333...
console.log(a % b);  // 1
console.log(a ** b); // 1000 (10 to the power of 3)
a++; console.log(a); // 11
b--; console.log(b); // 2
```

- **Assignment Operators**: =, +=, -=, *=, /=, %=, **=
```
let x = 5;
x += 3; // x = x + 3; // x is now 8
console.log(x); // 8
```

- **Comparison Operators**: == (loose equality), === (strict equality), !=, !==, >, <, >=, <=
```
console.log(5 == '5');   // true (loose equality, type coercion)
console.log(5 === '5');  // false (strict equality, no type
```

```
coercion)
console.log(10 > 5);     // true
```

- **Logical Operators**: && (AND), || (OR), ! (NOT)
```
let isAdult = true;
let hasLicense = false;
console.log(isAdult && hasLicense); // false
console.log(isAdult || hasLicense); // true
console.log(!isAdult);              // false
```

- **Ternary Operator**: condition ? expr1 : expr2
```
let age = 18;
let status = (age >= 18) ? "Adult" : "Minor";
console.log(status); // Output: Adult
```

1.4 Control Flow

Control flow statements allow you to execute different blocks of code based on certain conditions.

- **if, else if, else**:
```
let temperature = 25;
if (temperature > 30) {
    console.log("It's hot outside!");
} else if (temperature > 20) {
    console.log("It's pleasant.");
} else {
    console.log("It's cold.");
}
// Output: It's pleasant.
```

- **switch**:
```
let day = "Monday";
switch (day) {
    case "Monday":
        console.log("Start of the week.");
        break;
    case "Friday":
        console.log("End of the work week!");
        break;
    default:
        console.log("Just another day.");
}
// Output: Start of the week.
```

1.5 Loops

Loops are used to execute a block of code repeatedly.

- **for loop**:

```
for (let i = 0; i < 5; i++) {
    console.log("Iteration " + i);
}
/*
Output:
Iteration 0
Iteration 1
Iteration 2
Iteration 3
Iteration 4
*/
```

- **while loop**:

```
let count = 0;
while (count < 3) {
    console.log("Count: " + count);
    count++;
}
/*
Output:
Count: 0
Count: 1
Count: 2
*/
```

- **do...while loop**: Executes the block once, then checks the condition.

```
let i = 0;
do {
    console.log("Do-while iteration: " + i);
    i++;
} while (i < 0); // Condition is false, but runs once.
// Output: Do-while iteration: 0
```

- **for...in loop**: Iterates over enumerable properties of an object.

```
const car = { brand: "Toyota", model: "Camry", year: 2020 };
for (let key in car) {
    console.log(`${key}: ${car[key]}`);
}
/*
Output:
brand: Toyota
model: Camry
year: 2020
*/
```

- **for...of loop (ES6)**: Iterates over iterable objects (like Arrays, Strings, Maps, Sets, etc.).

```
const colors = ["red", "green", "blue"];
for (let color of colors) {
    console.log(color);
}
/*
Output:
red
green
blue
*/
```

1.6 Functions

Functions are reusable blocks of code that perform a specific task.

- **Function Declaration**:
```
function greet(name) {
    return "Hello, " + name + "!";
}
console.log(greet("Alice")); // Output: Hello, Alice!
```

- **Function Expression**:
```
const sayHi = function(name) {
    return "Hi, " + name + "!";
};
console.log(sayHi("Bob")); // Output: Hi, Bob!
```

- **Arrow Functions (ES6)**: Shorter syntax, no this binding, cannot be used as constructors.
```
const add = (a, b) => a + b;
console.log(add(5, 3)); // Output: 8

const multiply = (a, b) => {
    // Multi-line arrow function needs return
    return a * b;
};
console.log(multiply(4, 2)); // Output: 8
```

- **Parameters and Arguments**:
```
function calculateSum(num1, num2) { // num1, num2 are parameters
    return num1 + num2;
}
console.log(calculateSum(10, 20)); // 10, 20 are arguments
```

- **Default Parameters (ES6)**:
```
function welcome(name = "Guest") {
    console.log(`Welcome, ${name}!`);
}
welcome("Charlie"); // Output: Welcome, Charlie!
```

```
welcome();          // Output: Welcome, Guest!
```

2. Intermediate Concepts

2.1 Arrays

Arrays are ordered lists of values.

- **Creating Arrays**:
  ```
  let fruits = ["Apple", "Banana", "Cherry"];
  let emptyArray = [];
  let mixedArray = [1, "hello", true, { key: "value" }];
  ```

- **Accessing Elements**:
  ```
  console.log(fruits[0]); // Output: Apple
  console.log(fruits[fruits.length - 1]); // Output: Cherry
  ```

- **Modifying Elements**:
  ```
  fruits[1] = "Blueberry";
  console.log(fruits); // Output: ["Apple", "Blueberry", "Cherry"]
  ```

- **Common Array Methods**:
  - push(): Adds element to the end.
  - pop(): Removes element from the end.
  - unshift(): Adds element to the beginning.
  - shift(): Removes element from the beginning.
  - splice(start, deleteCount, ...items): Adds/removes elements at any position.
  - slice(start, end): Returns a shallow copy of a portion of an array.
  - concat(): Joins arrays.
  - indexOf(), lastIndexOf(), includes(): Search for elements.
  - forEach(), map(), filter(), reduce(): Iteration and transformation.
  - find(), findIndex(): Find elements based on a condition.

<!-- end list -->
```
let numbers = [1, 2, 3, 4, 5];
numbers.push(6);        // [1, 2, 3, 4, 5, 6]
numbers.pop();          // [1, 2, 3, 4, 5]
numbers.unshift(0);     // [0, 1, 2, 3, 4, 5]
numbers.shift();        // [1, 2, 3, 4, 5]
numbers.splice(2, 1, 10); // Removes 3, adds 10: [1, 2, 10, 4, 5]
let subArray = numbers.slice(1, 3); // [2, 10]

numbers.forEach(num => console.log(num * 2)); // Prints 2, 4, 20, 8,
10
let doubled = numbers.map(num => num * 2); // [2, 4, 20, 8, 10]
let evens = numbers.filter(num => num % 2 === 0); // [2, 10, 4]
let sum = numbers.reduce((acc, curr) => acc + curr, 0); // 22
```

2.2 Objects

Objects are collections of key-value pairs.

- **Creating Objects**:

```
let user = {
    name: "Jane Doe",
    age: 25,
    email: "jane@example.com",
    isAdmin: false,
    address: {
        street: "123 Main St",
        city: "Anytown"
    },
    greet: function() {
        console.log(`Hello, my name is ${this.name}`);
    }
};
```

- **Accessing Properties**:

```
console.log(user.name);          // Dot notation: Jane Doe
console.log(user['age']);        // Bracket notation: 25
console.log(user.address.city); // Anytown
user.greet();                    // Hello, my name is Jane Doe
```

- **Modifying/Adding Properties**:

```
user.age = 26;
user.phone = "555-1234";
console.log(user);
```

- **Deleting Properties**:

```
delete user.isAdmin;
console.log(user);
```

- **Object Destructuring (ES6)**:

```
const { name, age } = user;
console.log(name, age); // Jane Doe 26

const { city } = user.address;
console.log(city); // Anytown
```

- **Spread Operator (...) for Objects (ES2018)**:

```
const defaults = { color: "red", size: "M" };
const userPrefs = { size: "L", theme: "dark" };
const combined = { ...defaults, ...userPrefs };
console.log(combined); // { color: "red", size: "L", theme: "dark"
}
```

2.3 Prototypes and Prototypal Inheritance

JavaScript uses prototypal inheritance. Every object has a prototype, which is another object that it inherits properties and methods from.

```
const animal = {
    eats: true,
    walk() {
        console.log("Animal walks.");
    }
};

const rabbit = {
    jumps: true,
    __proto__: animal // rabbit inherits from animal
};

console.log(rabbit.eats); // Output: true (inherited from animal)
rabbit.walk();            // Output: Animal walks. (inherited from
animal)

// Using Object.create()
const dog = Object.create(animal);
dog.barks = true;
console.log(dog.eats); // true
```

2.4 this keyword

The this keyword refers to the object it belongs to. Its value depends on how the function is called.

- **Global context**: this refers to the global object (window in browsers, global in Node.js).
- **Method context**: this refers to the object the method belongs to.
- **Function context**: In non-strict mode, this refers to the global object. In strict mode, this is undefined.
- **Event handlers**: this refers to the element that received the event.
- **Arrow functions**: this is lexically scoped (it inherits this from the parent scope).
- **call(), apply(), bind()**: Explicitly set this.

<!-- end list -->

```
const myObject = {
    value: 42,
    getValue: function() {
        console.log(this.value); // `this` refers to myObject
    }
};
myObject.getValue(); // Output: 42

const anotherObject = {
```

```
    value: 100
};
// Using call to set `this` explicitly
myObject.getValue.call(anotherObject); // Output: 100

// Arrow function example
const arrowObject = {
    name: "Arrow Guy",
    sayName: function() {
        // `this` here refers to arrowObject
        const innerArrow = () => {
            console.log(this.name); // `this` here also refers to
arrowObject (lexical scope)
        };
        innerArrow();
    }
};
arrowObject.sayName(); // Output: Arrow Guy
```

2.5 Closures

A closure is the combination of a function and the lexical environment within which that function was declared. It allows a function to access variables from its outer scope even after the outer function has finished executing.

```
function createCounter() {
    let count = 0; // This variable is "closed over" by the returned
functions
    return {
        increment: function() {
            count++;
            console.log(count);
        },
        decrement: function() {
            count--;
            console.log(count);
        },
        getCount: function() {
            return count;
        }
    };
}

const counter1 = createCounter();
counter1.increment(); // Output: 1
counter1.increment(); // Output: 2
console.log(counter1.getCount()); // Output: 2
```

```
const counter2 = createCounter(); // New independent counter
counter2.increment(); // Output: 1
```

2.6 Higher-Order Functions

Functions that take other functions as arguments or return functions as their result.

```
// Function that takes a function as an argument
function operateOnNumbers(a, b, operation) {
    return operation(a, b);
}

function add(x, y) { return x + y; }
function subtract(x, y) { return x - y; }

console.log(operateOnNumbers(10, 5, add));     // Output: 15
console.log(operateOnNumbers(10, 5, subtract)); // Output: 5

// Function that returns a function
function multiplier(factor) {
    return function(number) {
        return number * factor;
    };
}

const multiplyBy5 = multiplier(5);
console.log(multiplyBy5(10)); // Output: 50
```

2.7 Event Loop and Asynchronous JavaScript

JavaScript is single-threaded, but it handles asynchronous operations (like network requests, timers) using the Event Loop.

- **Callbacks**: Functions passed as arguments to be executed later.

  ```
  console.log("Start");
  setTimeout(function() {
      console.log("Inside setTimeout (after 2 seconds)");
  }, 2000);
  console.log("End");
  // Output: Start -> End -> Inside setTimeout (after 2 seconds)
  ```

- **Promises (ES6)**: Objects representing the eventual completion or failure of an asynchronous operation.

  ```
  const fetchData = new Promise((resolve, reject) => {
      // Simulate an async operation
      setTimeout(() => {
          const success = true;
          if (success) {
  ```

```
            resolve("Data fetched successfully!");
        } else {
            reject("Failed to fetch data.");
        }
    }, 1500);
});

fetchData
    .then(data => console.log(data)) // Output: Data fetched
successfully!
    .catch(error => console.error(error))
    .finally(() => console.log("Promise finished."));
```

- **Async/Await (ES2017)**: Syntactic sugar built on Promises, making asynchronous code look synchronous.

```
async function getUserData() {
    try {
        console.log("Fetching user data...");
        const response = await
fetch('https://jsonplaceholder.typicode.com/users/1');
        const data = await response.json();
        console.log("User data:", data);
    } catch (error) {
        console.error("Error fetching user:", error);
    }
}
getUserData();
```

Note: fetch API is a Web API, not part of core JavaScript, but commonly used with async/await.

2.8 Error Handling

The try...catch...finally statement is used to handle errors gracefully.

```
function divide(a, b) {
    try {
        if (b === 0) {
            throw new Error("Cannot divide by zero!");
        }
        return a / b;
    } catch (error) {
        console.error("An error occurred:", error.message);
        return null; // Or re-throw, or return a default value
    } finally {
        console.log("Division attempt finished.");
    }
}

console.log(divide(10, 2)); // Output: Division attempt finished. \n 5
```

```
console.log(divide(10, 0)); // Output: An error occurred: Cannot
divide by zero! \n Division attempt finished. \n null
```

3. Advanced Concepts

3.1 ES6+ Features (Modern JavaScript)

- **Classes**: Syntactic sugar over JavaScript's existing prototypal inheritance.

```
class Animal {
    constructor(name) {
        this.name = name;
    }
    speak() {
        console.log(`${this.name} makes a sound.`);
    }
}

class Dog extends Animal {
    constructor(name, breed) {
        super(name); // Call parent constructor
        this.breed = breed;
    }
    speak() {
        console.log(`${this.name} (${this.breed}) barks.`);
    }
}

const myDog = new Dog("Buddy", "Golden Retriever");
myDog.speak(); // Output: Buddy (Golden Retriever) barks.
```

- **Modules (import/export)**: Organize code into separate files for better maintainability and reusability.
  - math.js

```
// math.js
export const PI = 3.14159;
export function add(a, b) {
    return a + b;
}
export default function subtract(a, b) {
    return a - b;
}
```

  - app.js

```
// app.js
import { PI, add } from './math.js';
import subtractNumbers from './math.js'; // Default import
```

```
console.log(PI); // 3.14159
console.log(add(2, 3)); // 5
console.log(subtractNumbers(5, 2)); // 3
```

    - *Note: Modules require a server environment or specific browser configuration (e.g., <script type="module">).*
- **Spread (...) and Rest (...) Operators**:
    - **Spread**: Expands an iterable (like an array or string) into individual elements.

```
const arr1 = [1, 2];
const arr2 = [3, 4];
const combinedArray = [...arr1, ...arr2, 5]; // [1, 2, 3, 4, 5]

const obj1 = { a: 1, b: 2 };
const obj2 = { c: 3, d: 4 };
const combinedObject = { ...obj1, ...obj2 }; // { a: 1, b: 2, c: 3, d: 4 }

function sum(x, y, z) { return x + y + z; }
const numbers = [1, 2, 3];
console.log(sum(...numbers)); // 6
```

    - **Rest**: Collects remaining arguments into an array.

```
function collectArgs(first, ...rest) {
    console.log(first); // 1
    console.log(rest);  // [2, 3, 4, 5]
}
collectArgs(1, 2, 3, 4, 5);
```

- **Template Literals (Template Strings)**: Backticks (`) for embedded expressions and multi-line strings.

```
const name = "World";
const greeting = `Hello, ${name}!
This is a multi-line string.`;
console.log(greeting);
```

- **Destructuring Assignment**: Extract values from arrays or properties from objects into distinct variables.

```
// Array Destructuring
const [first, second, ...restOfArray] = [10, 20, 30, 40, 50];
console.log(first); // 10
console.log(second); // 20
console.log(restOfArray); // [30, 40, 50]

// Object Destructuring (already covered)
```

- **Default Parameters**: (Already covered in Functions)

3.2 Generators

Functions that can be paused and resumed, yielding (returning) multiple values over time. They are defined with function*.

```
function* idGenerator() {
    let id = 1;
    while (true) {
        yield id++; // Pause execution and return id, resume on next()
    }
}

const gen = idGenerator();
console.log(gen.next().value); // 1
console.log(gen.next().value); // 2
console.log(gen.next().value); // 3
```

3.3 Proxies and Reflect (ES6)

- **Proxies**: Objects that wrap another object or function and can intercept fundamental operations (e.g., property lookup, assignment, function invocation).

```
const target = {
    message1: "hello",
    message2: "world"
};

const handler = {
    get: function(target, property, receiver) {
        if (property === 'message2') {
            return 'Intercepted message2!';
        }
        return Reflect.get(target, property, receiver);
    },
    set: function(target, property, value, receiver) {
        if (property === 'message1' && typeof value !== 'string')
{
            throw new TypeError('message1 must be a string');
        }
        return Reflect.set(target, property, value, receiver);
    }
};

const proxy = new Proxy(target, handler);

console.log(proxy.message1); // hello
console.log(proxy.message2); // Intercepted message2!
```

```
proxy.message1 = "new hello";
console.log(proxy.message1); // new hello

// proxy.message1 = 123; // Throws TypeError
```

- **Reflect**: A built-in object that provides methods for interceptable JavaScript operations. Often used with Proxies.

3.4 Web APIs (Browser Environment)

These are not part of core JavaScript but are crucial for web development.

- **DOM Manipulation (Document Object Model)**: Interface for HTML and XML documents.

```
// Select elements
const myDiv = document.getElementById('myDiv');
const paragraphs = document.querySelectorAll('.my-paragraph');

// Create elements
const newElement = document.createElement('p');
newElement.textContent = "This is a new paragraph.";
newElement.classList.add('dynamic-text');

// Append elements
document.body.appendChild(newElement);

// Modify attributes/styles
myDiv.style.backgroundColor = 'lightblue';
myDiv.setAttribute('data-info', 'important');

// Event Listeners
const myButton = document.getElementById('myButton');
myButton.addEventListener('click', () => {
    // In a real app, use a custom modal or message box instead of
alert()
    // For this example, we'll use a placeholder for brevity.
    console.log('Button clicked!');
});
```

- **Fetch API**: For making network requests (e.g., to APIs).

```
// Already demonstrated in Async/Await section.
// fetch(url, options) returns a Promise.
```

- **Local Storage and Session Storage**: Store data in the browser.
    - localStorage: Data persists even after browser close.
    - sessionStorage: Data cleared when the browser tab/window is closed. <!-- end list -->

```
// Local Storage
localStorage.setItem('username', 'Alice');
const username = localStorage.getItem('username');
```

```
console.log(username); // Alice
localStorage.removeItem('username');
// localStorage.clear(); // Clears all items

// Session Storage
sessionStorage.setItem('tempData', JSON.stringify({ id: 1, value:
'temporary' }));
const tempData = JSON.parse(sessionStorage.getItem('tempData'));
console.log(tempData); // { id: 1, value: 'temporary' }
```

3.5 Design Patterns

Common solutions to recurring problems in software design.

- **Module Pattern**: Encapsulate private state and public methods.

```
const ShoppingCart = (function() {
    let items = []; // Private variable

    function addItem(item) {
        items.push(item);
        console.log(`${item} added.`);
    }

    function removeItem(item) {
        const index = items.indexOf(item);
        if (index > -1) {
            items.splice(index, 1);
            console.log(`${item} removed.`);
        }
    }

    function getItems() {
        return [...items]; // Return a copy to prevent external
modification
    }

    return {
        add: addItem,
        remove: removeItem,
        list: getItems
    };
})();

ShoppingCart.add("Laptop");
ShoppingCart.add("Mouse");
console.log(ShoppingCart.list()); // ["Laptop", "Mouse"]
// console.log(ShoppingCart.items); // undefined (private)
```

- **Revealing Module Pattern**: A variation of the module pattern where you explicitly return an object with pointers to the private functions you want to expose.

```
const Calculator = (function() {
    let result = 0;

    function add(num) {
        result += num;
    }

    function subtract(num) {
        result -= num;
    }

    function getResult() {
        return result;
    }

    return {
        add: add,
        subtract: subtract,
        getResult: getResult
    };
})();

Calculator.add(10);
Calculator.subtract(3);
console.log(Calculator.getResult()); // 7
```

- **Singleton Pattern**: Ensures a class has only one instance and provides a global point of access to it.

```
const Logger = (function() {
    let instance;

    function init() {
        let logs = [];
        function addLog(message) {
            const timestamp = new Date().toISOString();
            logs.push(`[${timestamp}] ${message}`);
        }
        function getLogs() {
            return logs;
        }
        return {
            addLog: addLog,
            getLogs: getLogs
        };
    }
```

```
    return {
        getInstance: function() {
            if (!instance) {
                instance = init();
            }
            return instance;
        }
    };
})();

const logger1 = Logger.getInstance();
logger1.addLog("First message.");

const logger2 = Logger.getInstance();
logger2.addLog("Second message.");

console.log(logger1.getLogs()); // Both messages are present, as
it's the same instance.
console.log(logger2.getLogs());
```

- **Observer Pattern (Publish/Subscribe)**: Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.

```
class Subject {
    constructor() {
        this.observers = [];
    }

    addObserver(observer) {
        this.observers.push(observer);
    }

    removeObserver(observer) {
        this.observers = this.observers.filter(obs => obs !==
observer);
    }

    notify(data) {
        this.observers.forEach(observer => observer.update(data));
    }
}

class Observer {
    constructor(name) {
        this.name = name;
    }

    update(data) {
```

```
            console.log(`${this.name} received update: ${data}`);
        }
    }

    const newsFeed = new Subject();
    const subscriber1 = new Observer("Subscriber A");
    const subscriber2 = new Observer("Subscriber B");

    newsFeed.addObserver(subscriber1);
    newsFeed.addObserver(subscriber2);

    newsFeed.notify("Breaking News: JavaScript is awesome!");
    // Output:
    // Subscriber A received update: Breaking News: JavaScript is
    awesome!
    // Subscriber B received update: Breaking News: JavaScript is
    awesome!

    newsFeed.removeObserver(subscriber1);
    newsFeed.notify("Another update!");
    // Output:
    // Subscriber B received update: Another update!
```

3.6 Web Workers

Allows running JavaScript in the background thread, separate from the main execution thread, preventing UI blocking.

```
// main.js (or script in HTML)
// This part would typically be in your main HTML script
/*
if (window.Worker) {
    const myWorker = new Worker('worker.js'); // Assuming worker.js is
a separate file

    myWorker.postMessage({ command: 'startCalculation', data:
1000000000 });

    myWorker.onmessage = function(e) {
        console.log('Message received from worker:', e.data);
    };

    myWorker.onerror = function(error) {
        console.error('Worker error:', error);
    };

    console.log('Main thread continues...');
} else {
```

```
    console.log('Web Workers are not supported in this browser.');
}
*/

// worker.js content (this would be in a separate file named
worker.js)
/*
onmessage = function(e) {
    if (e.data.command === 'startCalculation') {
        let sum = 0;
        for (let i = 0; i < e.data.data; i++) {
            sum += i;
        }
        postMessage(sum); // Send result back to main thread
    }
};
*/
```

Note: Web Workers cannot directly access the DOM. The above code snippets for Web Workers are illustrative and would typically reside in separate files (main.js and worker.js) in a real project.

4. Real-World Project: Simple To-Do List Application

This project will demonstrate several concepts:

- DOM Manipulation
- Event Handling
- Local Storage for persistence
- Basic Array methods
- Functions

The application will allow users to add, delete, and mark tasks as complete, with tasks persisting across browser sessions.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Simple To-Do List</title>
    <!-- Tailwind CSS CDN for easy styling -->
    <script src="https://cdn.tailwindcss.com"></script>
    <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;600;700&
display=swap" rel="stylesheet">
    <style>
        body {
            font-family: 'Inter', sans-serif;
```

```
            background-color: #f0f2f5;
            display: flex;
            justify-content: center;
            align-items: flex-start; /* Align to top for better
content flow */
            min-height: 100vh;
            padding: 20px;
            box-sizing: border-box;
        }
        .container {
            background-color: #ffffff;
            border-radius: 1rem; /* Rounded corners */
            box-shadow: 0 10px 25px rgba(0, 0, 0, 0.1);
            padding: 2rem;
            width: 100%;
            max-width: 500px;
            margin-top: 2rem; /* Add some margin from the top */
        }
        .task-item {
            display: flex;
            align-items: center;
            justify-content: space-between;
            padding: 0.75rem 0;
            border-bottom: 1px solid #e5e7eb;
        }
        .task-item:last-child {
            border-bottom: none;
        }
        .task-text {
            flex-grow: 1;
            margin-right: 1rem;
            word-break: break-word; /* Ensure long words wrap */
        }
        .task-text.completed {
            text-decoration: line-through;
            color: #6b7280;
        }
        .action-buttons button {
            padding: 0.4rem 0.8rem;
            border-radius: 0.5rem;
            font-size: 0.875rem;
            font-weight: 600;
            transition: background-color 0.2s ease-in-out;
        }
        .action-buttons .complete-btn {
            background-color: #d1fae5; /* Light green */
            color: #065f46; /* Dark green text */
        }
```

```
.action-buttons .complete-btn:hover {
    background-color: #a7f3d0;
}
.action-buttons .delete-btn {
    background-color: #fee2e2; /* Light red */
    color: #991b1b; /* Dark red text */
    margin-left: 0.5rem;
}
.action-buttons .delete-btn:hover {
    background-color: #fecaca;
}
.input-group {
    display: flex;
    gap: 0.5rem;
    margin-bottom: 1.5rem;
}
.input-group input {
    flex-grow: 1;
    padding: 0.75rem 1rem;
    border: 1px solid #d1d5db;
    border-radius: 0.75rem;
    outline: none;
    transition: border-color 0.2s;
}
.input-group input:focus {
    border-color: #3b82f6; /* Blue focus ring */
}
.input-group button {
    background-color: #3b82f6; /* Blue */
    color: white;
    padding: 0.75rem 1.5rem;
    border-radius: 0.75rem;
    font-weight: 600;
    transition: background-color 0.2s ease-in-out;
    cursor: pointer;
}
.input-group button:hover {
    background-color: #2563eb; /* Darker blue */
}
.message-box {
    background-color: #fff;
    border-radius: 0.75rem;
    box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);
    padding: 1rem;
    margin-top: 1rem;
    text-align: center;
    position: fixed;
    top: 50%;
```

```
            left: 50%;
            transform: translate(-50%, -50%);
            z-index: 1000;
            display: none; /* Hidden by default */
            max-width: 300px;
        }
        .message-box button {
            background-color: #3b82f6;
            color: white;
            padding: 0.5rem 1rem;
            border-radius: 0.5rem;
            margin-top: 0.75rem;
            cursor: pointer;
        }
        .message-box.show {
            display: block;
        }
    </style>
</head>
<body class="antialiased">
    <div class="container">
        <h1 class="text-3xl font-bold text-center text-gray-800
mb-6">My To-Do List</h1>

        <div class="input-group">
            <input type="text" id="taskInput" placeholder="Add a new
task..." class="focus:ring-2 focus:ring-blue-500">
            <button id="addTaskBtn">Add Task</button>
        </div>

        <ul id="taskList" class="divide-y divide-gray-200">
            <!-- Tasks will be dynamically added here -->
        </ul>
    </div>

    <!-- Custom Message Box -->
    <div id="messageBox" class="message-box">
        <p id="messageBoxText"></p>
        <button id="messageBoxCloseBtn">OK</button>
    </div>

    <script>
        // Get DOM elements
        const taskInput = document.getElementById('taskInput');
        const addTaskBtn = document.getElementById('addTaskBtn');
        const taskList = document.getElementById('taskList');
        const messageBox = document.getElementById('messageBox');
        const messageBoxText =
```

```
document.getElementById('messageBoxText');
        const messageBoxCloseBtn =
document.getElementById('messageBoxCloseBtn');

        // Array to store tasks
        // Each task object will have: { id: string, text: string,
completed: boolean }
        let tasks = [];

        /**
         * Displays a custom message box instead of alert().
         * @param {string} message - The message to display.
         */
        function showMessageBox(message) {
            messageBoxText.textContent = message;
            messageBox.classList.add('show');
        }

        /**
         * Hides the custom message box.
         */
        function hideMessageBox() {
            messageBox.classList.remove('show');
        }

        // Event listener for the message box close button
        messageBoxCloseBtn.addEventListener('click', hideMessageBox);

        /**
         * Loads tasks from local storage when the page loads.
         */
        function loadTasks() {
            const storedTasks = localStorage.getItem('tasks');
            if (storedTasks) {
                tasks = JSON.parse(storedTasks);
                renderTasks();
            }
        }

        /**
         * Saves the current tasks array to local storage.
         */
        function saveTasks() {
            localStorage.setItem('tasks', JSON.stringify(tasks));
        }

        /**
         * Renders all tasks from the `tasks` array to the DOM.
```

```
         */
        function renderTasks() {
            taskList.innerHTML = ''; // Clear existing list items

            if (tasks.length === 0) {
                taskList.innerHTML = '<li class="text-center
text-gray-500 py-4">No tasks yet. Add one!</li>';
                return;
            }

            tasks.forEach(task => {
                const listItem = document.createElement('li');
                listItem.classList.add('task-item');
                listItem.setAttribute('data-id', task.id); // Store
task ID on the element

                // Task text span
                const taskTextSpan = document.createElement('span');
                taskTextSpan.classList.add('task-text',
'text-gray-700', 'text-lg');
                taskTextSpan.textContent = task.text;
                if (task.completed) {
                    taskTextSpan.classList.add('completed');
                }

                // Action buttons container
                const actionButtonsDiv =
document.createElement('div');
                actionButtonsDiv.classList.add('action-buttons');

                // Complete button
                const completeBtn = document.createElement('button');
                completeBtn.classList.add('complete-btn');
                completeBtn.textContent = task.completed ? 'Undo' :
'Complete';
                completeBtn.addEventListener('click', () =>
toggleComplete(task.id));

                // Delete button
                const deleteBtn = document.createElement('button');
                deleteBtn.classList.add('delete-btn');
                deleteBtn.textContent = 'Delete';
                deleteBtn.addEventListener('click', () =>
deleteTask(task.id));

                // Append elements
                actionButtonsDiv.appendChild(completeBtn);
                actionButtonsDiv.appendChild(deleteBtn);
```

```
                listItem.appendChild(taskTextSpan);
                listItem.appendChild(actionButtonsDiv);
                taskList.appendChild(listItem);
            });
        }

        /**
         * Adds a new task to the list.
         */
        function addTask() {
            const taskText = taskInput.value.trim(); // Get text and
remove whitespace

            if (taskText === '') {
                showMessageBox('Task cannot be empty!');
                return;
            }

            const newTask = {
                id: crypto.randomUUID(), // Generate a unique ID for
the task
                text: taskText,
                completed: false
            };

            tasks.push(newTask);
            saveTasks();      // Save to local storage
            renderTasks();    // Re-render the list
            taskInput.value = ''; // Clear input field
        }

        /**
         * Toggles the 'completed' status of a task.
         * @param {string} id - The ID of the task to toggle.
         */
        function toggleComplete(id) {
            tasks = tasks.map(task =>
                task.id === id ? { ...task, completed: !task.completed
} : task
            );
            saveTasks();
            renderTasks();
        }

        /**
         * Deletes a task from the list.
         * @param {string} id - The ID of the task to delete.
         */
```

```
        function deleteTask(id) {
            tasks = tasks.filter(task => task.id !== id);
            saveTasks();
            renderTasks();
        }

        // Event listener for Add Task button click
        addTaskBtn.addEventListener('click', addTask);

        // Event listener for Enter key press in the input field
        taskInput.addEventListener('keypress', (event) => {
            if (event.key === 'Enter') {
                addTask();
            }
        });

        // Load tasks when the page first loads
        document.addEventListener('DOMContentLoaded', loadTasks);
    </script>
</body>
</html>
```